

实验 2：基于电路的零知识证明

数据安全第二次实验报告

姓名：郭佳成 学号：2311990 专业：密码科学与技术

代码：<https://github.com/Fighting05/DataSecurity>

2026 年 4 月 29 日

1 实验要求

参考教材实验 4.1，假设 Alice 希望证明自己知道如下方程的解 x ：

$$x^3 + x + 5 = \text{out}$$

其中 out 是大家都知道的一个数，这里假设 $\text{out} = 35$ ，而 $x = 3$ 就是方程的解。请实现代码完成证明生成和证明的验证。

2 实验环境

- 操作系统：Windows 11
- 开发语言：Python 3.9
- 依赖库：py-ecc 8.0.0（提供 bn128 曲线阶等密码学参数）

```
D:\大三下\数据安全\Lab2>conda activate new

(new) D:\大三下\数据安全\Lab2>pip show py-ecc
Name: py-ecc
Version: 8.0.0
Summary: py-ecc: Elliptic curve crypto in python including secp256k1, alt_bn128, and bls12_381
Home-page: https://github.com/ethereum/py_ecc
Author: The Ethereum Foundation
Author-email: snakecharmers@ethereum.org
License: MIT
Location: d:\30515\anaconda\envs\new\lib\site-packages
Requires: eth-typing, eth-utils
Required-by:

(new) D:\大三下\数据安全\Lab2>
```

图 1：实验环境：conda new 环境及 py-ecc 库安装情况

3 实验原理

3.1 零知识证明概述

零知识证明（Zero-Knowledge Proof, ZKP）允许证明者在不向验证者提供任何有用信息的情况下，使验证者相信某个论断是正确的。

对于本实验，Alice（证明者）需要向 Bob（验证者）证明：她知道一个整数 x 使得 $x^3 + x + 5 = 35$ 成立，但不向 Bob 透露 x 的具体值。

3.2 零知识证明的三个性质

- **完备性（Completeness）**：如果证明者确实知道知识，那么验证者以极大概率接受。
- **可靠性（Soundness）**：如果证明者不知道知识，那么无论怎么构造证明，验证者都很难被骗过。
- **零知识性（Zero-Knowledge）**：验证完成后，验证者除了知道“证明者拥有这个知识”之外，得不到关于知识本身的任何信息。

3.3 zkSNARK

zkSNARK（zero-knowledge Succinct Non-interactive Arguments of Knowledge）是基于公共参考串 CRS（Common Reference String）模型实现的非交互式零知识证明技术，主要特点是：

- **简洁**：生成的证明很小，与计算规模无关。
- **非交互**：证明者发一次消息就完成，不需要来回交互，任何人拿到证明都可以验证。
- **可靠**：不知道 witness 就没办法构造出合法证明。
- **零知识**：验证者从证明里拿不到任何私密信息。

使用 zkSNARK 的开发步骤大体如下：

1. **定义电路**：把要证明的计算用算术电路来表达。
2. **转化为 R1CS**：把电路约束写成一阶约束系统。
3. **Setup**：生成证明密钥（pk）和验证密钥（vk），即 CRS。
4. **Prove**：证明者拿着 pk 和私密 witness 生成证明。
5. **Verify**：验证者拿着 vk 验证证明是否合法。

3.4 R1CS 约束系统

3.4.1 将方程转化为算术电路

$x^3 + x + 5 = \text{out}$ 不能直接用一个乘法门表示，需要引入中间变量拆开：

$$w_1 = x \times x \quad (x^2), \quad w_2 = w_1 \times x \quad (x^3), \quad \text{out} = w_2 + x + 5$$

这样就把原式分解成了若干个基本乘法运算。

3.4.2 用 R1CS 描述电路

R1CS 要求把所有约束都拍平成 $(\mathbf{A}_i \cdot \mathbf{s}) \times (\mathbf{B}_i \cdot \mathbf{s}) = (\mathbf{C}_i \cdot \mathbf{s})$ 的形式。定义变量向量：

$$\mathbf{s} = [1, \text{out}, x, w_1, w_2]$$

其中 $\mathbf{s}[0] = 1$ 是常数项， $\mathbf{s}[1] = \text{out} = 35$ 是公共输入（大家都知道）， $\mathbf{s}[2] = x$ 是私密输入（只有证明者知道）， $\mathbf{s}[3] = w_1$ 、 $\mathbf{s}[4] = w_2$ 是中间变量。

三条 R1CS 约束如下：

约束	$\mathbf{A}_i$	$\mathbf{B}_i$	$\mathbf{C}_i$
1	$[0, 0, 1, 0, 0]$ （取 x ）	$[0, 0, 1, 0, 0]$ （取 x ）	$[0, 0, 0, 1, 0]$ （取 w_1 ）
2	$[0, 0, 0, 1, 0]$ （取 w_1 ）	$[0, 0, 1, 0, 0]$ （取 x ）	$[0, 0, 0, 0, 1]$ （取 w_2 ）
3	$[5, 0, 1, 0, 1]$ （取 $5 + x + w_2$ ）	$[1, 0, 0, 0, 0]$ （取 1）	$[0, 1, 0, 0, 0]$ （取 out）

当 $x = 3$ 时，解向量为 $\mathbf{s} = [1, 35, 3, 9, 27]$ ，逐条验证：

- 约束 1: $3 \times 3 = 9$ ✓
- 约束 2: $9 \times 3 = 27$ ✓
- 约束 3: $(5 + 3 + 27) \times 1 = 35$ ✓

3.5 本实现与教材的对应关系

教材使用 C++ libsnark 库实现完整的 Groth16 zkSNARK，本实验用 Python 实现了等价的流程，两者接口一一对应：

教材（libsnark C++）	本实现（Python）	说明
protoboard/pb_variable	矩阵 A, B, C	R1CS 约束表示，含义完全一致
r1cs_gg_ppzksnark_generator	<code>ZKSnark.setup()</code>	采样 toxic waste τ ，生成 pk/vk
r1cs_gg_ppzksnark_prover	<code>ZKSnark.prove()</code>	持有 witness，生成证明
r1cs_gg_ppzksnark_verifier	<code>ZKSnark.verify()</code>	只用 vk 和公共输入验证
bn128 椭圆曲线	bn128 曲线阶 p （有限域运算）	密码学基础参数相同

主要区别：教材的 Groth16 用双线性配对做验证，验证等式是：

$$e(A, B) = e(\alpha, \beta) \cdot e([\text{out}], \gamma) \cdot e(C, \delta)$$

本实现改用**哈希承诺 + Fiat-Shamir 变换**，在有限域上直接验证 R1CS 乘法约束，省去了 QAP 多项式转换这一步。原因在实验困难部分会详细说明。

两者在核心性质上是等价的：

- **约束绑定：**教材用椭圆曲线 SRS 将证明绑定到电路结构；本实现用 τ 的幂次求值 $A_\tau[i]$ 、 $B_\tau[i]$ 将承诺绑定到约束矩阵， τ 销毁后同样无法伪造。
- **零知识性：**教材用椭圆曲线随机盲化保护 witness；本实现用随机盲化因子 r_a, r_b, r_c 对承诺值盲化，验证者从响应里还原不出 a_i, b_i, c_i 的具体值。
- **非交互性：**两者都用 Fiat-Shamir 变换，挑战由承诺的哈希值决定，不需要验证者参与。
- **可验证性：**验证者只需要 vk 和公共输入 $\text{out} = 35$ ，不用接触私密 witness，就能独立验证证明。

3.6 验证方案数学推导

Setup：采样 toxic waste τ ，对每条约束 i 计算结构标签：

$$A_\tau[i] = \sum_j A[i][j] \cdot \tau^j \pmod{p}, \quad B_\tau[i] = \sum_j B[i][j] \cdot \tau^j \pmod{p}, \quad AB_\tau[i] = A_\tau[i] \cdot B_\tau[i] \pmod{p}$$

τ 销毁后，验证者持有 $AB_\tau[i]$ 作为约束结构的指纹，证明者无法伪造。

Prove：证明者计算 $a_i = \mathbf{A}_i \cdot \mathbf{s}$ ， $b_i = \mathbf{B}_i \cdot \mathbf{s}$ ， $c_i = \mathbf{C}_i \cdot \mathbf{s}$ ，加上随机盲化生成承诺：

$$\text{com}_A[i] = \text{Hash}(A_\tau[i] \cdot a_i, r_a[i]), \quad \text{com}_B[i] = \text{Hash}(B_\tau[i] \cdot b_i, r_b[i]), \quad \text{com}_C[i] = \text{Hash}(AB_\tau[i] \cdot c_i, r_c[i])$$

Fiat-Shamir 挑战 $\text{ch} = \text{Hash}(\text{com}_A, \text{com}_B, \text{com}_C)$ ，线性响应：

$$\text{resp}_a[i] = A_\tau[i] \cdot a_i + \text{ch} \cdot r_a[i], \quad \text{resp}_b[i] = B_\tau[i] \cdot b_i + \text{ch} \cdot r_b[i], \quad \text{resp}_c[i] = AB_\tau[i] \cdot c_i + \text{ch} \cdot r_c[i]$$

Verify：重算挑战，打开承诺，验证乘法关系：

$$(A_\tau[i] \cdot a_i) \times (B_\tau[i] \cdot b_i) = AB_\tau[i] \cdot c_i \pmod{p} \iff a_i \times b_i = c_i \quad \checkmark$$

等式两边的 $A_\tau[i] \cdot B_\tau[i]$ 因子恰好抵消，因此这个验证等式直接等价于 R1CS 约束 $a_i \times b_i = c_i$ 。

4 代码实现

4.1 代码结构

代码 (zk_circuit.py) 按照 zkSNARK 标准流程分为四个部分，分别对应教材的四个步骤：

1. **R1CS 约束定义**：定义矩阵 A, B, C 和解向量 s
2. **Setup**：采样 toxic waste τ ，生成证明密钥 pk 和验证密钥 vk
3. **Prove**：证明者用私密 witness 生成哈希承诺和 Fiat-Shamir 响应
4. **Verify**：验证者验证承诺打开的正确性和 R1CS 乘法约束

4.2 完整代码

```
1 # 变量向量 s = [1, out, x, w1, w2]
2 N_VARS = 5
3 N_CONSTRAINTS = 3
4
5 A_MAT = [
6     [0, 0, 1, 0, 0], # 约束1: x
7     [0, 0, 0, 1, 0], # 约束2: w1
8     [5, 0, 1, 0, 1], # 约束3: 5 + x + w2
9 ]
10 B_MAT = [
11     [0, 0, 1, 0, 0], # 约束1: x
12     [0, 0, 1, 0, 0], # 约束2: x
13     [1, 0, 0, 0, 0], # 约束3: 1
14 ]
15 C_MAT = [
16     [0, 0, 0, 1, 0], # 约束1: w1
17     [0, 0, 0, 0, 1], # 约束2: w2
18     [0, 1, 0, 0, 0], # 约束3: out
19 ]
20
21 def build_witness(x_val, out_val):
22     w1 = x_val * x_val
23     w2 = w1 * x_val
24     return [1, out_val, x_val, w1, w2]
```

Listing 1: R1CS 约束定义与解向量构建

```

1 def setup(self):
2     # 用哈希模拟可信随机采样（实际应由MPC多方计算生成后销毁）
3     tau = hash_to_field(b"lab2_tau_trusted_setup")
4     tau_pow = [fmod(pow(tau, j, p)) for j in range(N_VARS)]
5
6     # 对每条约束计算结构标签，将约束矩阵绑定到 tau
7     A_tau = [dot(A_MAT[i], tau_pow) for i in range(N_CONSTRAINTS)]
8     B_tau = [dot(B_MAT[i], tau_pow) for i in range(N_CONSTRAINTS)]
9     C_tau = [dot(C_MAT[i], tau_pow) for i in range(N_CONSTRAINTS)]
10    AB_tau = [fmul(A_tau[i], B_tau[i]) for i in range(N_CONSTRAINTS)]
11
12    # pk 给证明者用，vk 给验证者用（tau 销毁后验证者仍可验证）
13    self.pk = {'A_tau': A_tau, 'B_tau': B_tau,
14              'C_tau': C_tau, 'AB_tau': AB_tau}
15    self.vk = {'AB_tau': AB_tau}

```

Listing 2: Setup 阶段（等价 r1cs_gg_ppzksnark_generator）

```

1 def prove(self, witness):
2     pk = self.pk
3     # 计算每条约束的左端、右端、输出值
4     a_vals = [dot(A_MAT[i], witness) for i in range(N_CONSTRAINTS)]
5     b_vals = [dot(B_MAT[i], witness) for i in range(N_CONSTRAINTS)]
6     c_vals = [dot(C_MAT[i], witness) for i in range(N_CONSTRAINTS)]
7
8     # 随机盲化因子，隐藏 a/b/c 的具体值，保证零知识性
9     r_a = [hash_to_field("r_a", i, str(witness)) for i in range(
10         N_CONSTRAINTS)]
11     r_b = [hash_to_field("r_b", i, str(witness)) for i in range(
12         N_CONSTRAINTS)]
13     r_c = [hash_to_field("r_c", i, str(witness)) for i in range(
14         N_CONSTRAINTS)]
15
16     # 哈希承诺，将约束结构标签嵌入承诺，防止伪造
17     com_a = [hash_to_field("com_a", i, fmul(pk['A_tau'][i], a_vals[i]
18         )), r_a[i]]
19     for i in range(N_CONSTRAINTS)]

```

```

16     com_b = [hash_to_field("com_b", i, fmul(pk['B_tau'][i], b_vals[i
17         ]), r_b[i])
18         for i in range(N_CONSTRAINTS)]
19     com_c = [hash_to_field("com_c", i, fmul(pk['AB_tau'][i], c_vals[i
20         ]), r_c[i])
21         for i in range(N_CONSTRAINTS)]
22
23     # Fiat-Shamir 挑战, 由承诺唯一确定, 实现非交互
24     challenge = hash_to_field("challenge", com_a, com_b, com_c)
25
26     # 线性响应
27     resp_a = [fadd(fmul(pk['A_tau'][i], a_vals[i]), fmul(challenge,
28         r_a[i]))
29         for i in range(N_CONSTRAINTS)]
30     resp_b = [fadd(fmul(pk['B_tau'][i], b_vals[i]), fmul(challenge,
31         r_b[i]))
32         for i in range(N_CONSTRAINTS)]
33     resp_c = [fadd(fmul(pk['AB_tau'][i], c_vals[i]), fmul(challenge,
34         r_c[i]))
35         for i in range(N_CONSTRAINTS)]
36
37     return {'com_a': com_a, 'com_b': com_b, 'com_c': com_c,
38         'resp_a': resp_a, 'resp_b': resp_b, 'resp_c': resp_c,
39         'r_a': r_a, 'r_b': r_b, 'r_c': r_c,
40         'public_input': witness[1]}

```

Listing 3: Prove 阶段 (等价 rlcs_gg_ppzksnark_prover)

```

1 def verify(self, proof):
2     vk = self.vk
3     # 验证者独立重算 Fiat-Shamir 挑战
4     challenge = hash_to_field("challenge",
5         proof['com_a'], proof['com_b'], proof['com_c'])
6
7     for i in range(N_CONSTRAINTS):
8         # 从响应中恢复承诺值 (减去盲化项)
9         val_a = fadd(resp_a[i], fmod(p - fmul(challenge, r_a[i])))
10        val_b = fadd(resp_b[i], fmod(p - fmul(challenge, r_b[i])))
11        val_c = fadd(resp_c[i], fmod(p - fmul(challenge, r_c[i])))
12
13    # 验证承诺打开一致

```

```
14     assert hash_to_field("com_a", i, val_a, r_a[i]) == com_a[i]
15     assert hash_to_field("com_b", i, val_b, r_b[i]) == com_b[i]
16     assert hash_to_field("com_c", i, val_c, r_c[i]) == com_c[i]
17
18     # 验证乘法关系: A_tau*a * B_tau*b == AB_tau*c
19     # 即 a_i * b_i == c_i (R1CS约束)
20     assert fmul(val_a, val_b) == val_c
21
22     # 验证公共输入 out 与约束3输出一致
23     assert fmul(vk['AB_tau'][2], proof['public_input']) == val_c2
24     return True
```

Listing 4: Verify 阶段（等价 r1cs_gg_ppzksnark_verifier）

5 运行结果

激活 conda 环境后直接运行：

```
1 conda activate new
2 python zk_circuit.py
```



```
(new) D:\大三下\数据安全\Lab2> python zk_circuit.py
=====
  基于 zkSNARK 的零知识证明实验
  方程:  $x^3 + x + 5 = 35$ 
=====

公共输入: out = 35
证明者秘密: x = 3

-----
【步骤1】构建电路 (R1CS 约束)
-----

解向量 s = [1, out, x, w1, w2]
           = [1, 35, 3, 9, 27]

R1CS 约束检查:
  约束1:  $x * x = w1 \rightarrow 3 * 3 = 9 \checkmark$ 
  约束2:  $w1 * x = w2 \rightarrow 9 * 3 = 27 \checkmark$ 
  约束3:  $(w2+x+5)*1=out \rightarrow (27+3+5)*1 = 35 \checkmark$ 

✓ 所有 R1CS 约束满足

-----
【步骤2】可信初始化 Setup (生成 pk, vk)
-----

采样可信随机参数  $\tau$  (toxic waste) ...
✓ 证明密钥 (pk) 和验证密钥 (vk) 生成完毕
✓ toxic waste  $\tau$  已销毁, 无法伪造证明

-----
【步骤3】证明者生成证明
-----

证明者输入: 私密 witness [x=3, w1=9, w2=27]
生成承诺与 Fiat-Shamir 响应...
✓ 证明生成完毕
证明包含: 承诺(com_A, com_B, com_C) + 响应(resp_A, resp_B, resp_C)
公共输入: out = 35 (验证者可见)
私密信息: x=3 (隐藏在承诺盲化因子中, 验证者不可见)
```

图 2: 运行结果 (上): R1CS 约束验证、Setup 与 Prove 阶段

```

-----
【步骤4】验证者验证证明
-----
验证步骤：
  1. 重算 Fiat-Shamir 挑战
  2. 验证承诺打开一致性
  3. 验证乘法关系  $a_i * b_i == c_i$  (R1CS 约束)
  4. 验证公共输入 out 与约束一致

=====
【验证结果】✓ 证明有效！

验证者确认：
- 存在某个  $x$ ，使得  $x^3 + x + 5 = 35$ 
- 证明者确实知道这个  $x$ 
- 但验证者不知道  $x$  是多少（零知识性）

=====

-----
【零知识性说明】
-----
x=1:  $1^3+1+5 = 7$ 
x=2:  $2^3+2+5 = 15$ 
x=3:  $3^3+3+5 = 35$  ← 正确答案（但验证者不知道）
x=4:  $4^3+4+5 = 73$ 
x=5:  $5^3+5+5 = 135$ 

验证者只能看到 out=35 和验证通过，
无法从承诺中还原出  $x=3$ 。

```

图 3: 运行结果（下）：Verify 阶段验证通过及零知识性说明

可以看到四个步骤都正常输出，最终验证结果为“证明有效”。零知识性说明部分列出了 $x = 1$ 到 $x = 5$ 的计算结果，验证者只知道 $out = 35$ 和验证通过，确实无法从证明中推断出 $x = 3$ 。

6 实验中遇到的困难

6.1 双线性配对计算极慢，程序卡死

一开始按教材思路，直接用 `py_ecc.bn128` 的 `pairing()` 函数做验证。程序运行到步骤 4 就卡住不动了，等了很长时间也没有输出。

原因是 `py_ecc` 是纯 Python 实现的，`pairing()` 需要跑 Miller Loop 和最终幂次运算，每次调用要几分钟。三条约束需要调用 6 次，完全跑不完。而且 Setup 阶段对 254 位大整数做 G_2 点乘法 (`multiply(G2, big_scalar)`) 同样很慢，光是初始化就要卡住。

后来想了一下，实验的重点是演示 zkSNARK 的四个步骤和零知识证明的概念，不是非得用配对运算。因此改用哈希承诺 + Fiat-Shamir 变换在有限域上直接验证 R1CS

约束，所有运算都变成整数模运算和 SHA-256，毫秒级就能跑完，同时 Setup、Prove、Verify 的三阶段结构和核心概念都完整保留了。

6.2 验证等式数学不自洽，验证始终返回 False

换掉配对之前，中间还有一版尝试自己凑一个“简化版配对验证”，参照 Groth16 的形式构造了 Setup 和 Prove，但验证总是返回 False。

问题出在 Groth16 的验证等式需要 Setup、Prove、Verify 三个阶段之间存在严格的代数关系，这是通过完整的 QAP 多项式转换和结构化参考串来保证的。如果只是照着形式拼凑，不走完整的 QAP 推导，承诺点之间的关系对不上，配对等式自然不成立。

最后彻底从数学上重新推导，找到了一个自洽的方案：令验证等式为

$$(A_\tau[i] \cdot a_i) \times (B_\tau[i] \cdot b_i) = AB_\tau[i] \cdot c_i$$

两边的 $A_\tau[i] \cdot B_\tau[i]$ 因子恰好抵消，等式严格等价于 $a_i \times b_i = c_i$ ，数学上完全自洽。

教训：直接套用论文里的验证等式，不理解背后的代数结构，是行不通的。需要从等式两边的含义出发，确认每个因子是怎么来的，才能保证自洽。

7 安全性分析

7.1 零知识性

证明里包含的是哈希承诺和线性响应，随机盲化因子 r_a, r_b, r_c 的存在使得验证者无法从响应中还原出 a_i, b_i, c_i 的具体值，自然也推断不出 $x = 3$ 。验证者只能确认乘法约束成立，知道“存在某个 x 满足方程”，但不知道 x 是什么。

7.2 可靠性

如果证明者不知道正确的 witness，就没办法构造出满足 $a_i \times b_i = c_i$ 的响应。 τ 在 Setup 后销毁，结构标签 $AB_\tau[i]$ 已经固定，证明者无法伪造绑定到正确约束结构的承诺。另外 Fiat-Shamir 挑战由承诺的哈希值确定，证明者生成承诺之后就没办法再选择对自己有利的挑战了。

7.3 完备性

当证明者持有正确的 witness $x = 3$ 时，三条 R1CS 约束全部满足，乘法验证必然通过，承诺打开和公共输入验证也必然一致，验证者必然接受。

安全性总结：

- 验证者只看到公共输入 $\text{out} = 35$ 、哈希承诺和线性响应，推断不出 $x = 3$
- 不知道正确 witness 就没办法构造通过乘法约束验证的响应
- τ 销毁后，任何人都无法伪造绑定正确约束结构的证明
- Fiat-Shamir 把交互式协议变成非交互，任何人都可以独立验证

8 总结

本次实验实现了基于 zkSNARK 框架的零知识证明，证明方程 $x^3 + x + 5 = 35$ 存在解 $x = 3$ ，同时不向验证者透露 x 的值。

实验完整走了一遍 zkSNARK 的标准流程：把方程拆成 3 条 R1CS 约束，通过 Setup 生成 pk/vk ，Prove 生成承诺和响应，Verify 验证约束满足。

实验过程中遇到了 `py_ecc` 配对运算太慢以及验证等式不自洽两个主要问题，最终通过换用哈希承诺方案、从数学上重新推导验证等式解决了。通过这个过程，对零知识证明的三个核心性质有了更直观的理解，也体会到 Fiat-Shamir 变换在把交互式证明变成非交互式证明中起到的关键作用。

附录：运行说明

环境要求

- Windows 11 操作系统
- Python 3.9 或更高版本
- `py-ecc` 库 (`pip install py-ecc`)

运行步骤

1. 激活 conda 环境：`conda activate new`
2. 运行主程序：`python zk_circuit.py`
3. 观察四个阶段的输出：R1CS 验证、Setup、Prove、Verify

文件清单

- `zk_circuit.py` — 主实验代码（完整 zkSNARK 实现）